

Лабораторная работа 6
**Реализация алгоритмов для расчета особых точек
с использованием OpenMP и oneTBB.**

Цель работы. Изучение различных видов реализации алгоритмов для расчета особых точек (на основе детектора Харриса) с использованием технологий OpenMP и oneTBB.

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

1. Ознакомиться с теоретическими сведениями.
2. Ознакомиться с алгоритмом расчета особых точек.
3. Выполнить задания 1–2.
4. Подготовить отчетные материалы по лабораторной работе.
5. Защитить лабораторную работу перед преподавателем.

ЗАДАНИЯ

Задание 6.1. Разработайте консольное приложение, выполняющее вычисление особых точек и их визуализацию на изображении. При реализации используйте возможности OpenMP и Intel oneTBB. Необходимо выполнить два типа реализации шага поиска особых точек для визуализации: один с использованием классических контейнеров STL или массивов, другой с применением потокобезопасных контейнеров oneTBB для шага 4. В приложение должны загружаться файлы input_X.bmp и формироваться выходные файлы output_X_var.bmp, где var означает имя реализации алгоритма обработки (или ее номер).

Задание 6.2. Проведите экспериментальное исследование по обработке данных с разными реализациями (не менее **10 запусков** для каждой реализации, **в случае последовательного кода можно делать 3 запуска**). Для каждой отдельной функции или функционального блока¹ представьте расчет времени выполнения, результаты оформите в виде таблицы 6.1.

Таблица 6.1

Вид реализации функции	Поток	НД 1		НД 2		НД 3	
		Время	$S_p(n)$	Время	$S_p(n)$	Время	$S_p(n)$
Функция 1 реализации 1 (посл.)							
Функция 1 реализация (парал.) OpenMP*	2						
	3						
	4						

¹ Функциональные блоки представлены в виде выделенных шагов в разделе «Практические советы к реализации» в качестве функциональных блоков также выступают шаги 4.1 4.2 и 4.3

Продолжение таблицы 6.1

Вид реализации функции	Поток	НД 1		НД 2		НД 3	
		Время	$S_p(n)$	Время	$S_p(n)$	Время	$S_p(n)$
Функция 1 реализация (парал.) oneTBB*	2						
	3						
	4						
.....							
Функция К реализации 1 (посл.)							
Функция К реализация (парал.) OpenMP*	2						
	3						
	4						
Функция К реализация (парал.) oneTBB*	2						
	3						
	4						

* – возможно использование различных подходов для реализации распараллеливания вычислительных этапов (шагов) формирования карты особых точек.

Используя результаты эксперимента по каждой функции (данные таблицы 6.1) определить общее время формирования карты особых точек, рассчитать ускорение и оформить в виде таблицы 6.2.

Таблица 6.2

Вид реализации функции	Поток	НД 1		НД 2		НД 3	
		Время	$S_p(n)$	Время	$S_p(n)$	Время	$S_p(n)$
Реализация последов (посл.)							
OpenMP* (парал.) Шаг 4 STL контейнер или массив	2						
	3						
	4						
OpenMP* (парал.) Шаг 4 потокобезопасный контейнер oneTBB	2						
	3						
	4						
Intel oneTBB* (парал.) Шаг 4 STL контейнер или массив	2						
	3						
	4						
Intel oneTBB* (парал.) Шаг 4 потокобезопасный контейнер oneTBB	2						
	3						
	4						
Intel oneTBB* + OpenMP* (парал.) Шаг 4 STL контейнер или массив	2						
	3						
	4						
Intel oneTBB* + OpenMP* (парал.) Шаг 4 потокобезопасный контейнер oneTBB	2						
	3						
	4						

* – возможно использование различных подходов для реализации распараллеливания вычислительных этапов (шагов) формирования карты особых точек.

ОТЧЕТНЫЕ МАТЕРИАЛЫ

Отчетные материалы сдаются в электронном виде и включают:

- 1) файл отчета в формате (doc/docx/odt), который должен содержать:
 - титульный лист;
 - цель работы;
 - задания работы (без дополнительных пояснений и вида заполняемых таблиц);
 - конфигурация компьютера, на котором производится выполнение заданий и параметры операционной системы;
 - экранные снимки с демонстрацией работы программ согласно заданиям (фрагменты вывода информации в консоли);
 - примеры исходных и обработанных данных (картинки из задания 6.1 или 6.2);
 - выводы по результатам исследования отдельно по функциям на основе таблицы 6.1 и по всем шагам формирования карты особых точек (таблица 6.2), а также выводы по лабораторной работе в целом;
 - ответы на контрольные вопросы.
- 2) файл электронной таблицы в Excel совместимом формате (xls/xlsx/ods), который должен содержать результаты выполнения экспериментального исследования в табличной форме.
- 3) архив формата **zip** с исходными кодами заданий лабораторной работы (исходные коды решения, без результатов сборки).

КОНТРОЛЬНЫЕ ВОПРОСЫ И ЗАДАНИЯ

1. Охарактеризуйте oneTBV parallel_sort.
2. Расскажите о потокобезопасных контейнерах.
3. Охарактеризуйте oneTBV concurrent_vector.
4. Какие методы oneTBV concurrent_vector позволяют безопасно добавлять элементы и увеличивать размер?
5. Охарактеризуйте oneTBV concurrent_queue.
6. В чем заключается основное отличие concurrent_queue от queue при программной реализации доступа к элементам?
7. Поясните понятие mutex и приведите пример использования.
8. Охарактеризуйте известные вам особенности oneTBV mutex.

ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ ПО АЛГОРИТМУ РАСЧЕТА И ВИЗУАЛИЗАЦИИ ОСОБЫХ ТОЧЕК

Для выделения из изображения некоторой интерпретируемой информации необходимо привязаться к локальным особенностям изображения, в роли которых могут выступать особые точки. **Особая точка**, или точечная особенность (англ. point feature, key point, feature) изображения,

это точка изображения, окрестность которой можно отличить от окрестности любой другой точки изображения.

Существуют множество алгоритмов определения особых точек, предназначенных для разных областей применения. Наиболее часто встречаются алгоритмы, использующие детекторы углов (или как еще их называют уголковым детекторам)².

Углы (corners) – особые точки, которые формируются из двух или более граней, и грани обычно определяют границу между различными объектами и/или частями одного и того же объекта.

Широкое распространение получил детектор Харриса, который выделяет особенности посредством анализа приращений интенсивностей изображения (градиент) в окрестности анализируемого элемента разложения как в горизонтальном, так и в вертикальном направлениях. Для каждого пикселя изображения вычисляется значение функции отклика угла – оценивающая степень похожести изображения окрестности точки на угол.

Для нахождения значения функции отклика угла в детекторе Харриса вначале вычисляется матрица $\mathbf{M}$:

$$\mathbf{M} = \begin{bmatrix} \left(\frac{\partial I}{\partial x}\right)^2 & \left(\frac{\partial I}{\partial x}\right)\left(\frac{\partial I}{\partial y}\right) \\ \left(\frac{\partial I}{\partial x}\right)\left(\frac{\partial I}{\partial y}\right) & \left(\frac{\partial I}{\partial y}\right)^2 \end{bmatrix},$$

где $I(x,y)$ – функция яркости изображения в точке с координатами (x, y) ; $\partial I/\partial x$, $\partial I/\partial y$ – частные производные по переменным x , y .

Затем рассчитываются собственные значения матрицы $\mathbf{M}$. В случае, когда оба собственных значения матрицы велики, даже небольшое смещение точки (x, y) в сторону вызывает значительные изменения яркости, что и соответствует особенности изображения.

Для упрощения вычислений Харрис предложил вместо собственных значений матрицы рассчитывать функцию отклика угла:

$$R = \det \mathbf{M} - k(\text{trace}(\mathbf{M}))^2,$$

где $k = 0,04$ (коэффициент, предложенный Харрисом); $\det \mathbf{M}$ – определитель матрицы; $\text{trace}(\mathbf{M})$ – след матрицы $\mathbf{M}$ (сумма по диагонали матрицы).

Особенностями признаются точки изображения, соответствующие локальным максимумам функции R .

² Более подробно с видами детекторов можно ознакомиться в статье «Детекторы углов»: <http://www.pvsm.ru/algoritmy/76984>.

Практические советы к реализации

Решение поставленной задачи в рамках лабораторной работы 6 можно условно разбить на пять основных шагов.

Шаг 1. Формирование массива яркости изображения

Для хранения значений яркости изображения лучше сформировать массив типа float (возможно и double), при этом значения яркости вычислять по формуле:

$$I(y, x) = \text{Red}(y, x) \times 0,299 + \text{Green}(y, x) \times 0,587 + \text{Blue}(y, x) \times 0,114.$$

Для снижения влияния шумов при нахождении особых точек необходимо осуществить предварительную обработку изображения (например, применить сглаживающий фильтр Гаусса). Для сглаживания можно использовать любую понравившуюся реализацию фильтра Гаусса, полученную при выполнении предыдущей лабораторной работы.

Шаг 2. Вычисление первого и второго градиента яркости по горизонтали и вертикали

Целесообразно сформировать массивы вещественного типа для хранения значений градиентов:

DiffX[][] – градиент по горизонтали $\left(\frac{\partial I}{\partial x}\right)^2$;

DiffY[][] – градиент по вертикали $\left(\frac{\partial I}{\partial y}\right)^2$;

DiffXY[][] – градиент $\left(\frac{\partial I}{\partial x}\right)\left(\frac{\partial I}{\partial y}\right)$.

Получение градиента в пикселе (y, x) происходит на основе свертки с использованием формулы, аналогично той, что использовалась при реализации линейного фильтра³:

$$I_{new}(y, x) = \sum_{dy=-1}^1 \sum_{dx=-1}^1 K_{dy+1, dx+1} \times I_{old}(y + dy - 1, x + dx - 1).$$

Для каждого вида градиента характерны следующие коэффициенты:

– градиент по горизонтали $\left(\frac{\partial I}{\partial x}\right)$: $K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$;

³ За основу можно взять реализацию двумерного фильтра Гаусса, в который нужно подставлять соответствующие матрицы коэффициентов, однако не требуется коррекция результата в диапазон [0...255]

– градиент по вертикали $\left(\frac{\partial I}{\partial y}\right): \quad K = \begin{vmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{vmatrix};$

DiffX[] – рассчитывать как последовательное применение свертки по горизонтали два раза;

DiffY[] – рассчитывать как последовательное применение свертки по вертикали два раза;

DiffXY[] – рассчитывать как применение в начале свертки по горизонтали, потом к полученному результату свертки по вертикали.

!!! Внимание !!! После расчета отклика свертки в данном случае нет необходимости проверять значение на соответствия диапазону 0...255 как в случае линейных фильтров.

Заполнение массивов **DiffX** **DiffY** и **DiffXY** можно выполнить в рамках двух проходов, в ходе первого прохода выполняется свертка массива **I** по горизонтали и вертикали одновременно, в рамках чего значения записываются данные в массивы **TmpDiffX** и **TmpDiffY**. Далее используя эти массивы выполняется расчет итоговых значений для массивов **DiffX** **DiffY** и **DiffXY** путем одновременного расчета результата выполнения трех сверток. Обозначив матрицы коэффициентов для градиентов по горизонтали и вертикали как **MATR_coef_X** и **MATR_coef_Y** псевдокод можно представить следующим образом:

```
/* Первый проход */
Цикл 1 по Y (Y от 0 до Height)
Цикл 2 по X (X от 0 до Width)
  Начало 2
  LinF_Value_X = 0;
  LinF_Value_Y = 0;
  Цикл A1 (DY от -1 до 1 включительно)
    Начало A1
    KY = Y + DY
    Если KY < 0 то KY = 0
    Если KY > Height - 1 то KY = Height - 1
    Цикл A2 (DX от -1 до 1 включительно)
      Начало A2
      KX = X + DX
      Если KX < 0 то KX = 0
      Если KX > Width - 1 то KX = Width - 1
      LinF_Value_X += I[KY][KX] ×
                     MATR_coef_X[DY + 1][DX + 1]
      LinF_Value_Y += I[KY][KX] ×
                     MATR_coef_Y[DY + 1][DX + 1]
    Конец A2
  Конец A1
Конец 2
```

```

    Конец A1
    TmpDiffX[y][x]= LinF_Value_X
    TmpDiffY[y][x]= LinF_Value_Y
    Конец 2

/* Второй проход */
Цикл 3 по Y (Y от 0 до Height)
Цикл 4 по X (X от 0 до Width)
    Начало 4
    LinF_Value_XX =0;
    LinF_Value_YY =0;
    LinF_Value_XY =0;
    Цикл Б1 (DY от -1 до 1 включительно)
        Начало Б1
        KY = Y + DY
        Если KY < 0 то KY=0
        Если KY > Height - 1 то KY = Height -1
        Цикл Б2 (DX от -1 до 1 включительно)
            Начало Б2
            KX = X + DX
            Если KX < 0 то KX = 0
            Если KX > Width - 1 то KX = Width - 1
            LinF_Value_XX += TmpDiffX[KY][KX] ×
                           MATR_coef_X[DY + 1][DX + 1]
            LinF_Value_XY += TmpDiffX[KY][KX] ×
                           MATR_coef_Y[DY + 1][DX + 1]
            LinF_Value_YY += TmpDiffY[KY][KX] ×
                           MATR_coef_Y[DY + 1][DX + 1]

            Конец Б2
        Конец Б1
    DiffX[y][x]= LinF_Value_XX
    DiffY[y][x]= LinF_Value_YY
    DiffX[y][x]= LinF_Value_XY
    Конец 4

```

Шаг 3. Вычисление значения функции отклика угла R

Для хранения значений с результатами функция отклика угла R целесообразно сформировать двумерный массив H[[]]. Расчет значения функции отклика угла R происходит на основе матрицы M, которую с точки зрения практической реализации можно записать в следующем виде:

$$\mathbf{M} = \begin{bmatrix} \text{DiffX} & \text{DiffXY} \\ \text{DiffXY} & \text{DiffY} \end{bmatrix}.$$

С использованием обозначений:

$A = \text{DiffX}[y][x];$
 $B = \text{DiffY}[y][x];$
 $C = \text{DiffXY}[y][x].$

функция вычисления отклика угла **R** примет следующий вид:

$$R(y,x) = (A \times B - C \times C) - (k \times ((A + B) \times (A + B))).$$

Результат вычисления отклика угла **R** записать в массив **H**:

$H[y][x] = R(y, x);$

Шаг 4. Поиск особых точек для визуализации

Шаг 4.1. Формирование вспомогательного списка *HarrisPoints*⁴, содержащего данные *R*, *y*, *x*. В контейнер помещается информация об особых точках, у которых значение в массиве $H[y][x]$ больше значения определенного как порог (*threshold*).

Если $H[y][x] > \text{threshold}$ **то**

HarrisPoints.добавить ($H[y][x], y, x$) ;

Определение порога *threshold* происходит на основе данных о минимальном и максимальном значениях массива $H[][]$ по формуле:

$$\text{threshold} = (\max(H[][])) - \min(H[][])) \times kf + \min(H[][]), \quad kf \in [0.6, 0.8]^5$$

Шаг 4.2. Сортировка вспомогательного списка особых точек (контейнера *HarrisPoints*) на основе значения *R* по убыванию. Сортировку вспомогательного списка особых точек **HarrisPoints** можно осуществить, используя любую реализацию алгоритма сортировки из предыдущих лабораторных работ, а также используя сортировки `std::sort` или `TBB`.

Шаг 4.3. Отсечение точек по порогу и удаление точек с меньшим значением в заданной окрестности.

Возможна следующая реализация: после сортировки по убыванию, в цикле по списку выполняется проверка наличия особых точек в некоторой окрестности относительно анализируемой позиции. При этом для окрестности выбирается *rang*(радиус) $\text{rang} \geq 20$.

Если в контейнере **HarrisPoints** будет найдена точка в заданном радиусе, то она удаляется из контейнера, схематично данный алгоритм можно представить следующим образом:

Цикл 1 по *I* контейнера **HarrisPoints** (*I* от 0 до размера)

Начало 1

Цикл 2 по *J* контейнера **HarrisPoints** (*J* от размера до *I*+1)

Начало 2

⁴ может использоваться любой контейнер, например *vector*

⁵ Нахождение минимума и максимума реализовать в едином цикле до формирования вспомогательного списка *HarrisPoints*

```

Если Евклидово расстояние между HarrisPoints[I] и
HarrisPoints[J] < rang то
    Удалить из контейнера HarrisPoints[j]
Конец 2
Конец 1

```

Альтернативным подходом будет занесение в список **HarrisPointsVs** только тех точек, которые находятся на расстоянии большем, чем установленное значение радиуса проверки (rang) для всех точек, перечисленных в списке **HarrisPointsVs**.

```

HarrisPointsVs.добавить(HarrisPoints[0])
Цикл 1 по I контейнера HarrisPoints (I от 1 до размера)
Начало 1
    Флаг = 0
    Цикл 2 по J контейнера HarrisPointsVs (J от 0 до размера)
    Начало 2
        Если Евклидово расстояние между HarrisPoints[I] и
        HarrisPointsVs [J] < rang то
            Флаг++
    Конец 2
    Если Флаг = 0 то
        HarrisPointsVs.добавить(HarrisPoints[I])
Конец 1

```

Окрестность проверки **rang** задается пользователем (т. е. передается как параметр в функцию).

Шаг 5. Визуализация особых точек

В окрестности (rang=1) для особой точки в списке **HarrisPoints** (или альтернативном списке **HarrisPointsVs**) с координатами [y][x] в результирующем изображении нужно все закрасить красным цветом⁶, в противном случае данные о цвете взять из оригинального изображения⁷.

⁶ Возможен и другой цвет, но он должен выделяться на изображении

⁷ Можно сделать копию оригинального изображения, а потом, проходя в цикле по контейнеру, выполнять отрисовку особых точек